OPERAÇÕES COM ARQUIVOS EM DISCO

A linguagem C divide as categorias de acesso a disco em dois grandes grupos chamados de alto nível, ou leitura e gravação *bufferizada*, e de baixo-nível, ou leitura e gravação não *bufferizada*.

ALTO NÍVEL	BAIXO NÍVEL
Caractere: getc() ou fgetc(), putc() ou fputc()	read(), write()
String: fgets(), fputs()	
Formatada: fscanf(), fprintf()	
Registro: fread(), fwrite()	

Arquivos de Texto

Um arquivo aberto em modo texto é uma seqüência de caracteres agrupadas em linhas. As linhas são separadas pelo caractere de final de linha lógico ou LF, de código ASCII 10 decimal.

No DOS um arquivo aberto em modo texto são seqüências de caracteres agrupadas em linhas e essas são separadas por dois caracteres, o caractere 13 decimal, ou CR, e o caractere 10 decimal ou LF.

O compilador C converte o par CR/LF em um único caractere de final de linha quando um arquivo aberto em modo texto é lido, e converte o caractere de final de linha no par CR/LF quando o arquivo é gravado em disco.

Se um arquivo é aberto em modo texto, ele reconhece a indicação de fim de arquivo enviada pelo DOS.

Abrir um Arquivo

Para trabalhar com um arquivo de dados, o primeiro passo é estabelecer um *buffer,* onde as informações serão armazenadas temporariamente entre a memória do computador e o arquivo de dados. Esse buffer permite que informações sejam lidas de/ou escritas em arquivo de dados mais rapidamente.

```
FILE *fptr
```

FILE (maiúsculo) é uma estrutura pré definida FILE (no *header file* stdio.h) que estabelece o buffer e *fptr* é uma variável ponteiro que indica o início de buffer.

Para gerar um código de programa que abre um arquivo, o compilador precisa conhecer 3 aspectos:

1º O nome do arquivo que será usado
2º O tipo de abertura
3º Onde guardar informações sobre o arquivo.

```
fptr=fopen("arquivo.txt","w");
 ③              ①       ②
```

Abre um arquivo chamado "arquivo.txt" no disco corrente para escrita "w", a qual será armazenada na variável **fptr** declarada como ponteiro para o tipo FILE.

Quando é necessário colocar o caminho completo do arquivo, e a conseqüente presença de barras invertidas " **** " como em "c:\aluno\teste.txt", ao invés de colocar uma barra, deve-se colocar "****", portanto ficando assim "c:\\aluno\\teste.txt";

A função *fopen* retorna o valor NULL caso o arquivo não possa ser aberto.

Exemplos:

```
FILE *cliente;  //Definição do buffer cliente

FILE *fornecedor;  //Definição do buffer fornecedor

cliente = fopen("clientes.txt", "w");

fornecedor = fopen("c:\\aluno\\fornec.txt", "r");
```

Tipos de abertura de arquivo:

CÓD.	DESCRIÇÃO
"r".	Abrir um arquivo texto para leitura. O arquivo deve estar presente no disco
"w"	Abrir um arquivo texto para gravação. Se o arquivo estiver presente ele será destruído e reinicializado. Se não existir, ele será criado.
"a"	Abrir um arquivo texto para inclusão (acréscimo de informações). Os dados serão adicionados ao fim do arquivo existente, ou um novo arquivo será criado.
"r+"	Abrir um arquivo texto para leitura e gravação. O arquivo deve existir e pode ser atualizado.
"w+"	Abrir um arquivo texto para leitura e gravação. Se o arquivo existir ele será destruído e reinicializado. Se não existir, será criado.
"a+"	Abrir um arquivo texto para leitura e para adicionar dados ao fim do arquivo existente. Caso não exista, um novo arquivo será criado.
"rb"	Abrir um arquivo binário para leitura. O arquivo deve estar presente no disco.
"wb"	Abrir um arquivo binário para gravação. Se o arquivo estiver presente ele será destruído e reinicializado. Se não existir, ele será criado.
"ab"	Abrir um arquivo binário para inclusão. Os dados serão adicionados ao fim do arquivo existente, ou um novo arquivo será criado. Não permite leitura dos dados.
"rb+"	Abrir um arquivo binário para leitura e gravação. O arquivo deve existir e pode ser atualizado.
"wb+"	Abrir um arquivo binário para leitura e gravação. Se o arquivo existir ele será destruído e reinicializado. Se não existir, será criado.
"ab+"	Abrir um arquivo binário para atualizações e para adicionar dados ao fim do arquivo existente ou um novo arquivo será criado. Possibilita a leitura dos dados existentes.

Fechar um Arquivo - fclose()

Sintaxe: **fclose(** *nomebuf* **)**;

A função fclose é utilizada para fechar um determinado arquivo após sua utilização, é recomendado deixar o arquivo aberto somente quando for utilizá-lo. Quando a função fclose é utilizada qualquer caracter que estiver no buffer é gravado em disco evitando perda de dados.

Onde: ***nomebuf*** é o nome da variável que está sendo utilizada como ponteiro para o buffer do arquivo a ser fechado.

Verificando final de arquivo - EOF

EOF

EOF é a marca de fim de arquivo. É um constante inteira pré-definida no *header file* stdio.h, nos microcomputadores IBM-PC essa constante tem o valor -1. É utilizada quando a leitura é feita caractere a caractere, onde após cada uma das letras lidas pode-se verificar se já está no fim do arquivo, evitando erros no programa.

feof()

Sintaxe: *int* **feof (nomebuf)**

A função ***feof*** é utilizada para a verificação do final do arquivo. Essa função retorna um valor inteiro caso o ponteiro chegou no final do arquivo, caso contrário o retorno é zero. Essa função pode ser utilizada quando a leitura é feita caractere a caractere ou de qualquer outra forma

Onde: ***nomebuf*** é o nome da variável que está sendo utilizada como ponteiro para o buffer do arquivo onde será feita a verificação de final de arquivo.

Funções para Ler/Gravar caracter a caracter

Leitura Caracter a Caracter - getc() ou fgetc()

Sintaxe:

int **fgetc(** *nomebuf* **)**;

int **getc(** nomebuf **)**

As funções *fgetc* e *getc* são utilizadas para a leitura de caracteres de um arquivo texto, onde os caracteres são lidos um a um. As duas funções trabalham da mesma forma, é necessária a existência das duas para a compatibilidade entre sistemas. A função *getc* retorna o caracter lido do arquivo referenciado por ***nomebuf*** esse caracter pode ser armazenado em uma variável inteira ou caracter. É recomendado a utilização do inteiro para a comparação com a constante inteira **EOF.**

Onde:

nomebuf é o nome da variável que está sendo utilizada como ponteiro para o buffer do arquivo a ser lido.

varint é a variável inteira ou caracter que irá armazenar o dado lido.

Gravação Caracter a Caracter - putc() ou fputc()

Sintaxe:

fputc(caracter, *nomebuf* **)**;

putc(caracter**,** nomebuf **)**

As funções *fputc* e put*c* são utilizadas para a gravação de caracteres de um arquivo texto,

onde os caracteres são gravados um a um.

Onde:

nomebuf é o nome da variável que está sendo utilizada como ponteiro para o buffer do arquivo a ser gravado, lembrar que este arquivo deve ser aberto para gravação com w ou a.

caracter é a variável caracter que será armazenada no arquivos especificado por nomebuf.

Funções para Ler/Gravar Strings

Ler strings - fgets()

Sintaxe: char ***fgets(***stringlida, tamanho, *nomebuf* **)**;

A função *fgets* lê uma string do arquivo especificado por *nomebuf***.** Essa função lê os caracteres de um arquivos até que um final de linha seja encontrado, ou até que a string de retorno ***stringlida*** contenha (***tamanho***-1) caracteres. O retorno da função pode ser comparado com NULL para verificar se já é final de arquivo ou se ocorreu um erro durante a leitura.

Onde:

nomebuf é o nome da variável que está sendo utilizada como ponteiro para o buffer do arquivo a ser lido.

stringlida é a variável do tipo cadeia de caracteres ou um ponteiro para caracteres, onde serão armazenados os dados lidos do arquivo.

tamanho é a quantidade máxima de caracteres que serão lidos de uma vez.

Exemplo:

```
FILE *fornecedor;

char lido[80];

fornecedor = fopen("c:\\aluno\\fornec.txt", "r");

if (fgets(lido, 80, fornecedor) == NULL)

   printf("Ocorreu um erro na leitura do arquivo ou EOF");

else

   printf("%s", lido);
```

Gravar strings - fputs()

Sintaxe: **fputs(** stringgrava, *nomebuf* **)**;

A função *fputs* grava a string contida em ***stringgrava*** do arquivo especificado por ***nomebuf*****.**.O final de linha é colocado, ou não dependendo do ambiente utilizado.

Onde:

nomebuf é o nome da variável que está sendo utilizada como ponteiro para o buffer do

arquivo a ser gravado, não esquecendo que este arquivo deve ser aberto para a gravação.

stringgrava é a variável do tipo cadeia de caracteres ou um ponteiro para caracteres, onde será armazenada no arquivo.

Exemplo:

```
FILE *fornecedor;

char gravar[80];

fornecedor = fopen("c:\\aluno\\fornec.txt", "w");

gets(gravar);

fputs(gravar, fornecedor);
```

Funções para Ler/Gravar Strings Formatadas

Ler Strings Formatadas- fscanf()

Sintaxe: **fscanf(** *nomebuf***,** *formatação*, *variáveis* **)**;

A função ***fscanf*** lê uma string formatada do arquivo especificado por ***nomebuf*.** Essa função lê os caracteres de um arquivo de acordo com a formatação definida por ***formatação*** e os coloca nas variáveis definidas por ***variáveis.***

Onde:

nomebuf é o nome da variável que está sendo utilizada como ponteiro para o buffer do arquivo a ser lido.

formatação é uma string de formatação, definindo como os dados serão lidos do arquivo. A formatação é definida na mesma forma do scanf.

Gravar Formatadas- Gravar - fprintf()

Sintaxe: char ***fscanf(** *nomebuf***,** *formatação*, *variáveis* **)**;

A função ***fscanf*** lê uma string formatada do arquivo especificado por ***nomebuf*.** Essa função lê os caracteres de um arquivo de acordo com a formatação definida por ***formatação*** e os coloca nas variáveis definidas por ***variáveis.***

Onde:

nomebuf é o nome da variável que está sendo utilizada como ponteiro para o buffer do arquivo a ser lido.

formatação é uma string de formatação, definindo como os dados serão lidos do arquivo. A formatação é definida na mesma forma do *scanf*.

Funções para Ler/Gravar Estruturas

Estruturas- Ler - fread()

Sintaxe: **fread(***&estrutura,* **sizeof(***estrutura***),** *num_estr, nomebuf* **)**;

A função ***fread*** lê num_estr elementos gravados em disco, muito utilizado na leitura de estruturas gravadas em disco. A leitura dos dados depende do tipo de dados que está gravado, caso for um inteiro, ele irá ler o dado em dois bytes. Portanto um arquivo gravado com estruturas, inteiros, e qualquer outro dado que não seja caracter, será ilegível pelo DOS***.***

Onde:

estrutura é o nome da estrutura que será colocado o dado lido do disco. Antes do nome da estrutura deve-se colocar um & comercial, pois o fread necessita do endereço dessa estrutura.

sizeof(*estrutura****)*** é o tamanho em bytes que a estrutura a ser lida possui. Caso for um inteiro essa função retorna o valor 2.

num_estr é a quantidade de estruturas serão lidas de uma vez.

nomebuf é o nome da variável relacionada ao arquivo que será lido.

Gravar Estruturas- fwrite()

Sintaxe: **fwrite(***&estrutura,* **sizeof(***estrutura***),** *num_estr, nomebuf* **)**;

A função ***fwrite*** grava n elementos em disco, muito utilizado na gravação de estruturas em disco. A gravação dos dados depende do tipo de dados que está gravado, caso for um inteiro, ele irá gravar o dado em dois bytes do arquivo. Portanto um arquivo gravado com estruturas, inteiros, e qualquer outro dado que não seja caracter, será ilegível pelo DOS***.***

Onde:

estrutura é o nome da estrutura que contém os dados a serem gravados no disco. Antes do nome da estrutura deve-se colocar um & comercial, pois o fread necessita do endereço dessa estrutura.

sizeof(*estrutura****)*** é o tamanho em bytes que a estrutura a ser armazenada possui. Caso for um inteiro essa função retorna o valor 2.

num_estr é a quantidade de estruturas serão gravados de uma vez.

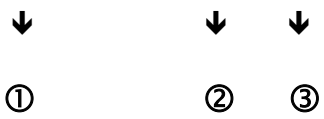
nomebuf é o nome da variável relacionada ao arquivo que será utilizado para a gravação das informações.

Acesso Aleatório

fseek()

Posiciona o ponteiro do arquivo na posição desejada.

Sintaxe: **fseek(**pont tipo FILE ,desl,modo)



1- ponteiro do tipo file da estrutura

2- faixa de deslocamento do arquivo, que consiste no número do registro a ser apontado vezes o tamanho da estrutura, a partir do 3º argumento.

3- determina a partir de onde o deslocamento começará a ser acessado

0 - SEEK_SET - começo do arquivo

1 - SEEK_CUR - posição corrente do arquivo

2 - SEEK_END - fim do arquivo

ftell()

Sintaxe: long **ftell**(pont. tipo FILE);

Retorna a posição do ponteiro de um arquivo binário em relação ao seu começo. Esta função aceita um único argumento, que é o ponteiro para a estrutura FILE do seu arquivo e retorna um valor do tipo long, que representa o número de bytes do começo do arquivo até a posição atual.

rewind(file)

Retorna o ponteiro do arquivo para o início do arquivo

Condições de Erro

ferror()

Determina se ocorreu algum erro na leitura/gravação, retorna não zero se ocorreu algum erro e 0 se nenhum erro ocorreu.

Sintaxe: int **ferror(**ponteiro do tipo FILE**)**

perror()

Imprime a mensagem do erro fornecida pelo sistema.

Sintaxe: **perror**(String);

Exemplo :

```
/* Tentativa de abrir para escrita um arquivo protegido */
FILE *fp;
fp=fopen("io.sys","r");
if (ferror(fp))
{
   printf("Arquivo não pode ser aberto");
   perror("Causa do Erro : ");
}
```

Resulta em :

```
Arquivo nao pode ser aberto
Causa do Erro : Permission Denied
```